

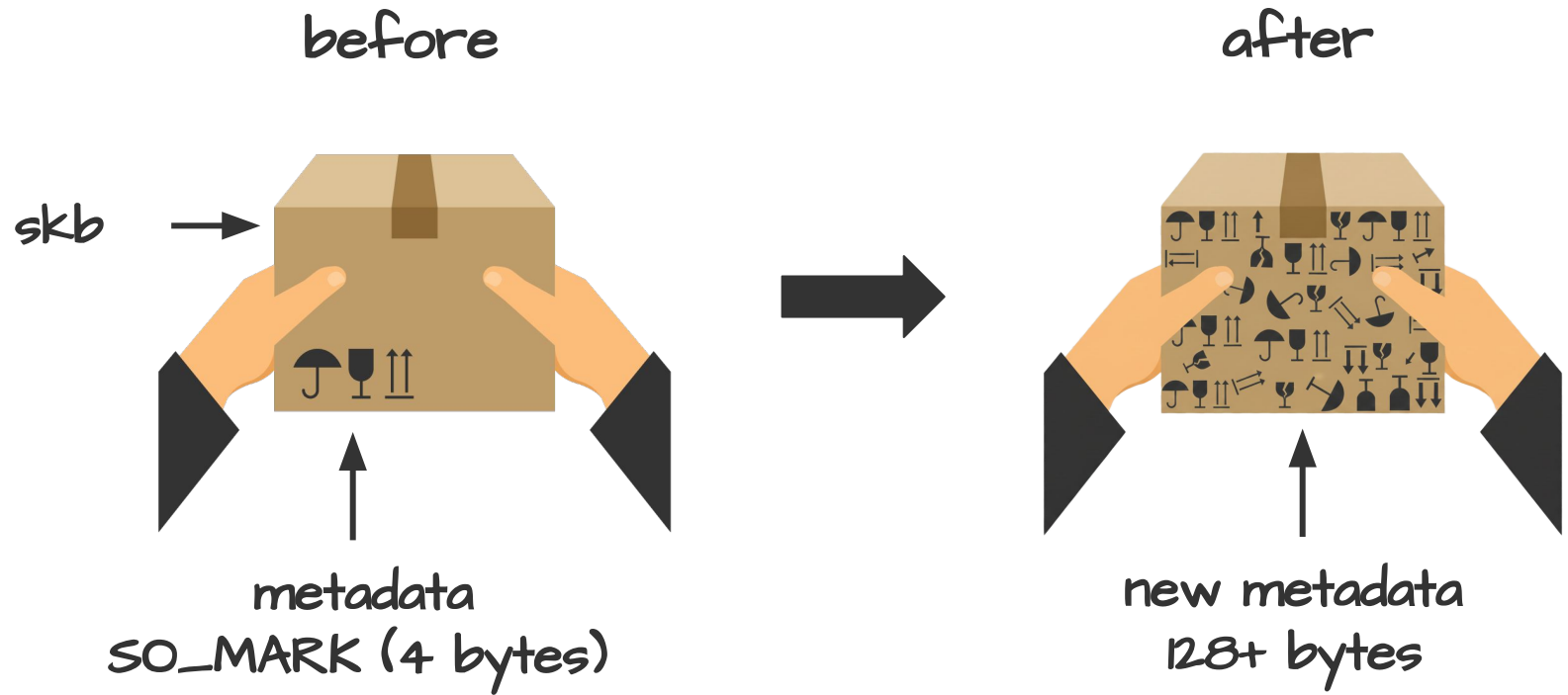


Following the breadcrumbs – Tracing packets through the kernel with per-skb BPF metadata

Jakub Sitnicki
Performance Engineer
Cloudflare

Linux System Monitoring and Observability MC
Linux Plumbers Conference 2026
Prague, Czech Republic

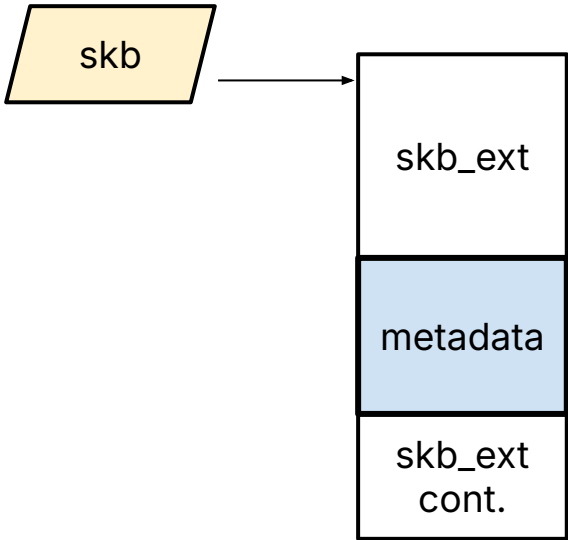
Continuation of an ongoing effort...



Continuation of an ongoing effort...



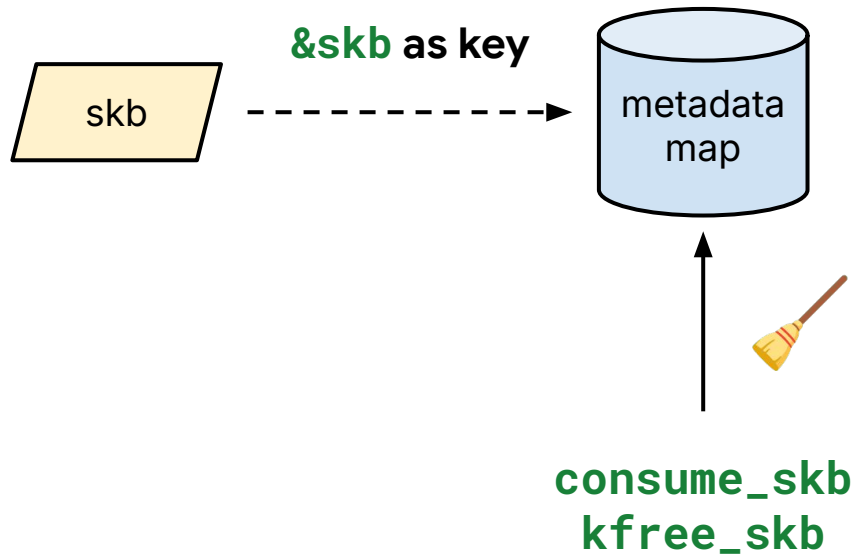
LSF/MM/BPF Summit 2026



Continuation of an ongoing effort...



Netdev Ox1A, 2026

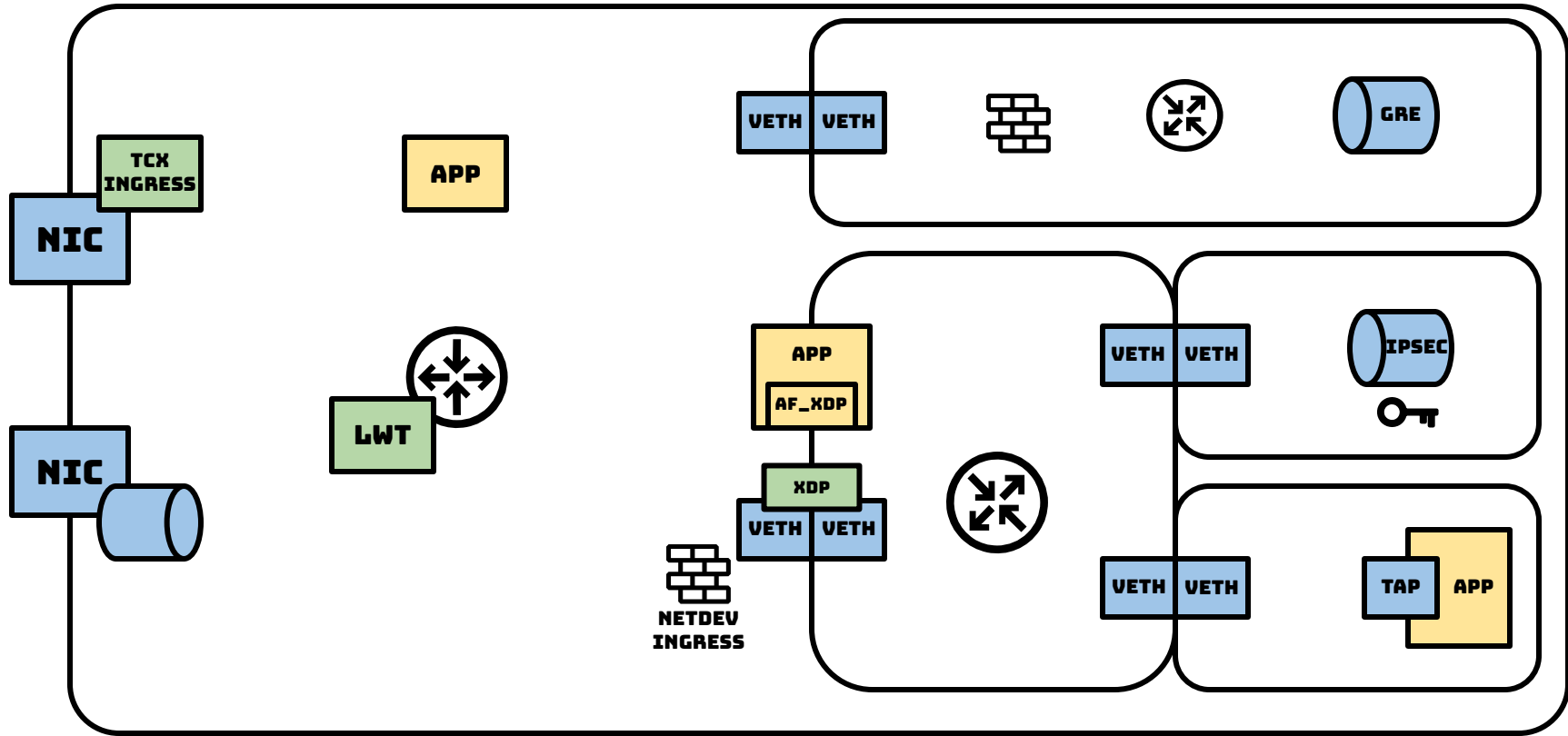


Agenda

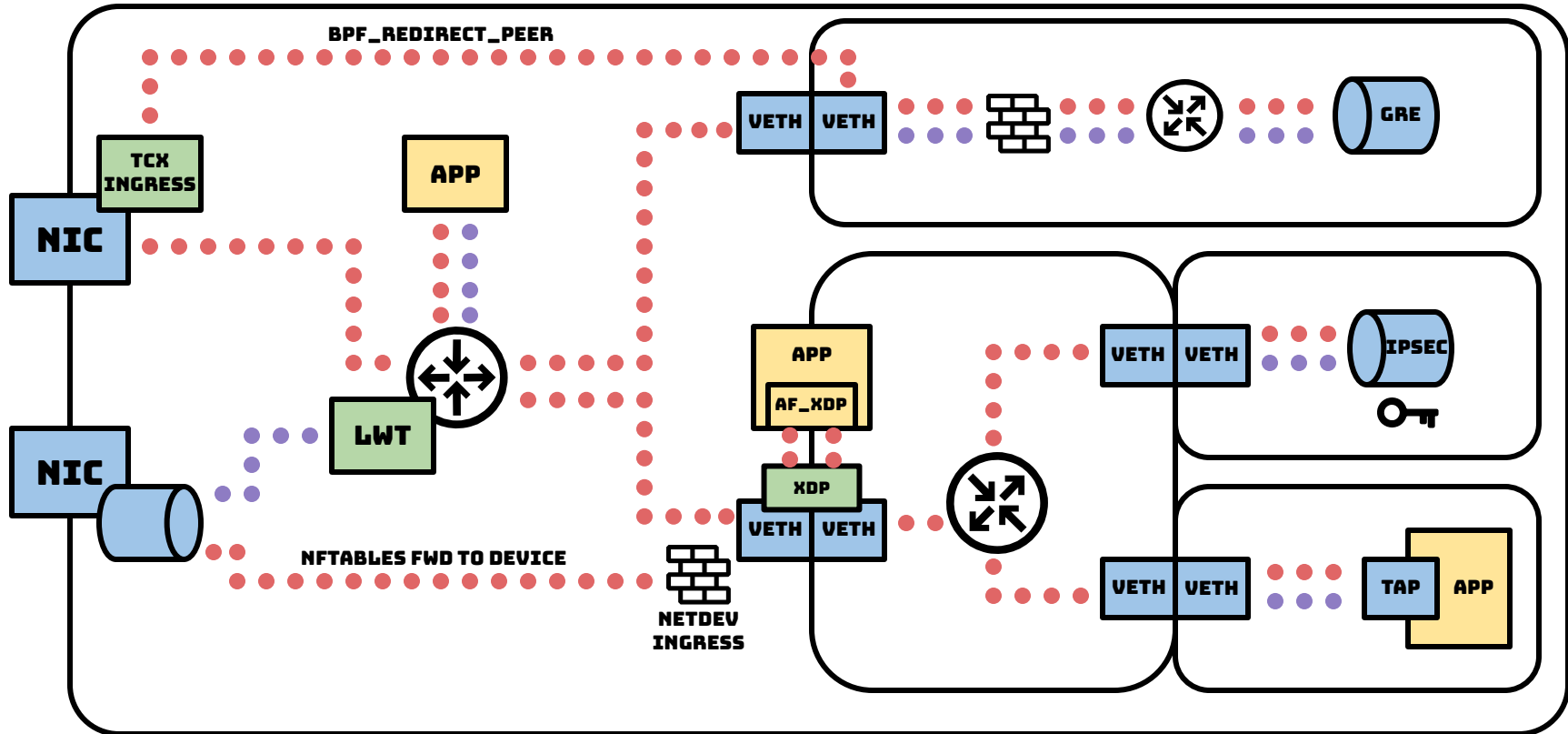
- 1 Per-skb BPF metadata from tracing PoV
- 2 Gated skb tracepoints



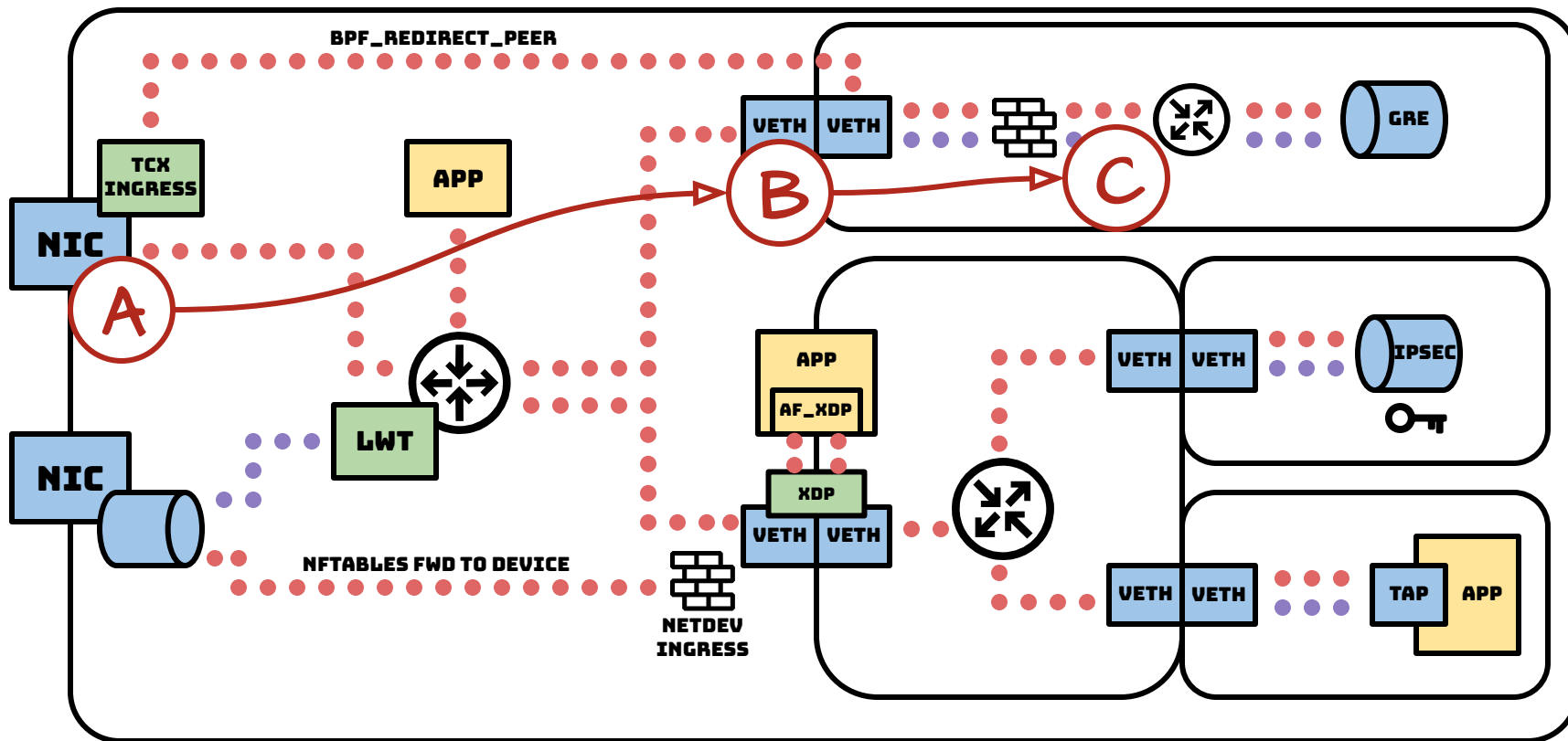
Situation – Networking setups be complex



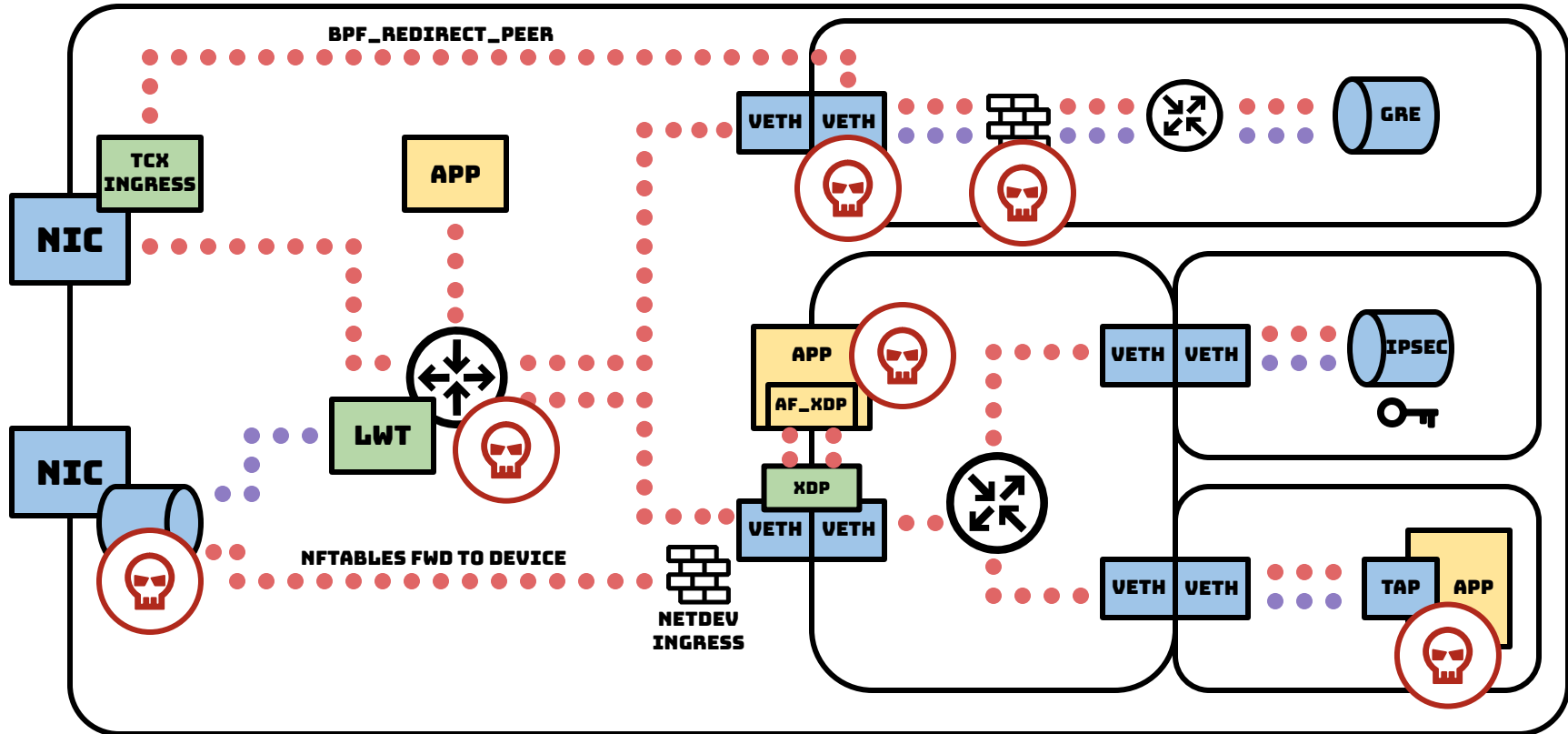
Problem – Guess which path your packet took?



Problem – Guess how long did it take from A to B to C? CLOUDFLARE



Problem – Guess where it got dropped?



Solution – Existing packet tracing tools



pwru (packet, where are you?)

<https://github.com/cilium/pwru>



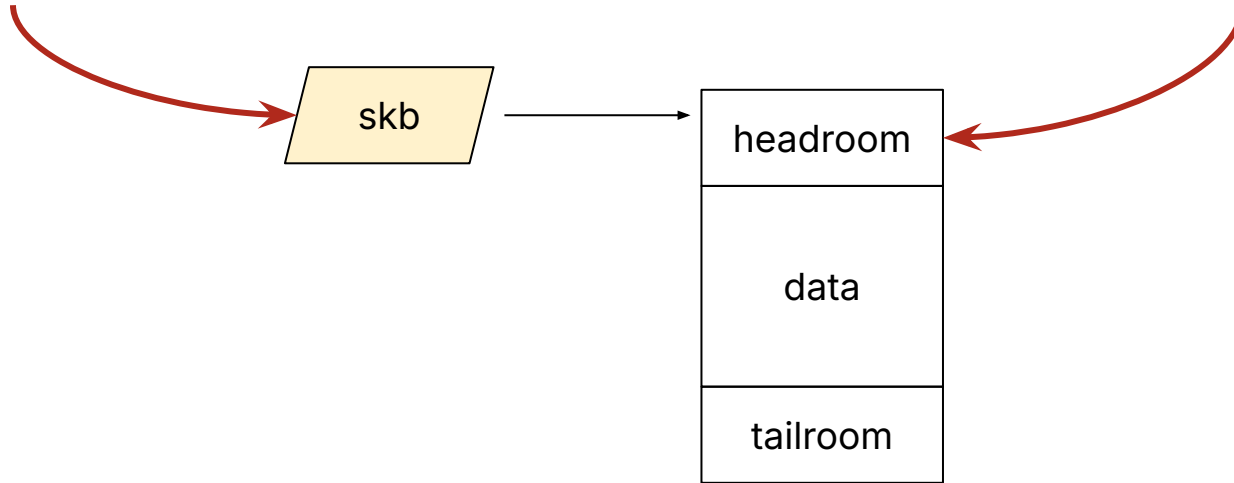
Retis

<https://github.com/retis-org/retis>

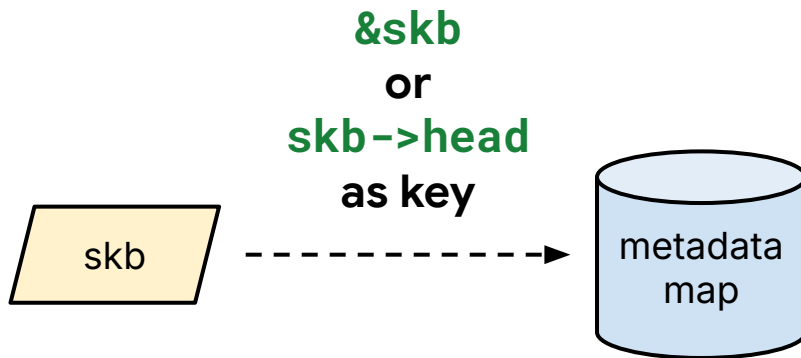
Difference in approach to packet tracing

pwru tracks `&skb`

retis tracks `skb->head`



... translates to difference in cleaning up metadata



If tracking by &skb

- `tracepoint:skb:consume_skb`
- `tracepoint:skb:kfree_skb`
- (... more if you want to track clones)

If tracking by skb->head

- `fentry:vmlinux:skb_free_head`
- `fentry:vmlinux:kfree_skb_partial`
- `fentry:vmlinux:pskb_expand_head`

... translates to runtime cost

```
jakub@116m100:~$ sudo bpftrace track_method_eval.bt
```

```
Attached 9 probes
```

BY SKB DATA	BY SKB ADDR	DELTA
-----	-----	-----
588703	505324	83379
629328	529976	99352
606493	513123	93370
620767	524488	96279
584516	490188	94328
588419	498555	89864
638577	539312	99265
681687	574227	107460
695844	608763	87081
626724	530230	96494

```
@by_skb_addr: 0
```

```
@by_skb_data: 0
```

```
jakub@116m100:~$
```



BPF tracepoint progs
will run **~15%** times more often
when tracking **skb->head**

A word about UI... we don't love it

```
$ sudo retis collect -p tp:net:netif_receive_skb -p net:net_dev_start_xmit \  
-c skb,ns,dev,skb-tracking -f 'icmp and ip dst host 192.0.2.2'  
L2+L3 packet filter(s) loaded  
5 probe(s) loaded  
69985404794126 (6) [ping] 817453 [tp] net:net_dev_start_xmit #3fa6bf31350e... (skb ffff8ab642032d00)  
192.0.2.1 > 192.0.2.2 tos 0x0 ttl 64 id 20907 off 0 [DF] len 84 proto ICMP (1) type 8 code 0  
ns 0x44/4026533699 if 6 (greA)  
  
69985404809449 (6) [ping] 817453 [tp] net:net_dev_start_xmit #3fa6bf31350e... (skb ffff8ab642032d00)  
10.0.0.1 > 172.16.0.1 tos 0x0 ttl 64 id 18957 off 0 [DF] len 108 proto GRE (47)  
ns 0x44/4026533699 if 5 (AB)  
  
69985404812955 (6) [ping] 817453 [tp] net:netif_receive_skb #3fa6bf31350e... (skb ffff8ab642032d00)  
10.0.0.1 > 172.16.0.1 ... proto GRE (47)  
ns 0x45/4026533797 if 5 (BA)  
  
69985404820048 (6) [ping] 817453 [tp] net:net_dev_start_xmit #3fa6bf31350e... (skb ffff8ab642032d00)  
10.0.0.1 > 172.16.0.1 ... proto GRE (47)  
ns 0x45/4026533797 if 6 (BC) rxif 5  
  
69985404821819 (6) [ping] 817453 [tp] net:netif_receive_skb #3fa6bf31350e... (skb ffff8ab642032d00)  
10.0.0.1 > 172.16.0.1 ... proto GRE (47)  
ns 0x46/4026533895 if 5 (CB)  
  
69985404829837 (6) [ping] 817453 [tp] net:netif_receive_skb #3fa6bf31350e... (skb ffff8ab642032d00)  
192.0.2.1 > 192.0.2.2 tos 0x0 ttl 64 id 20907 off 0 [DF] len 84 proto ICMP (1) type 8 code 0  
ns 0x46/4026533895 if 6 (greC)
```

Tracing
ping over
a GRE tunnel

We would like something like **mtr**

```
retis live - tracking #a1b2c3d4e5f6 (skb ffff8881234abcd0)
```

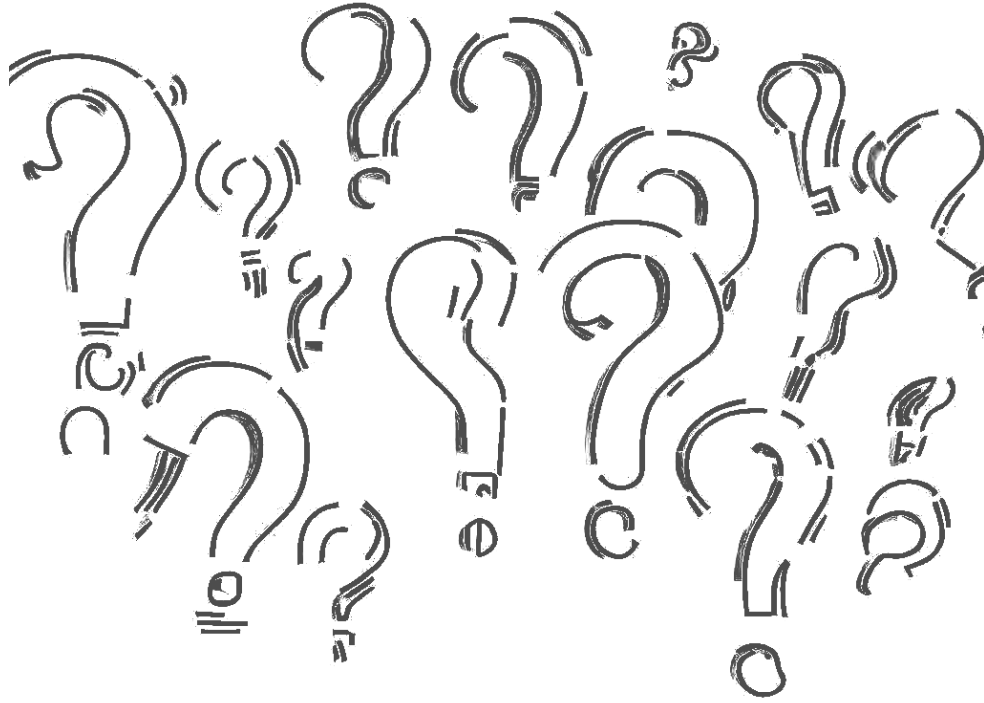
Step	Netns	Interface	Function/Tracepoint	Δt (μs)	Info
- 10.0.0.1:43210 → 10.0.0.2:80 TCP SYN len 64					
1	ns 1 (init)	eth0	rx netif_receive_skb	+0.0	
2	ns 1 (init)	eth0	rx ip_rcv	+1.2	
3	ns 1 (init)		nf_hook_slow	+3.4	nft: filter/FWD
4	ns 1 (init)	veth0	tx dev_queue_xmit	+5.1	
—— netns boundary ——					
5	ns 2 (ctr1)	eth0	rx netif_receive_skb	+6.8	
6	ns 2 (ctr1)		ip_forward	+8.0	
7	ns 2 (ctr1)	genev0	tx dev_queue_xmit	+9.2	
encap: Geneve vni 0x1a					
10.0.0.2:17145 → 10.0.0.3:6081 UDP len 106 geneve vni 0x1a					
inner: 10.0.0.1:43210 → 10.0.0.2:80 TCP SYN					
8	ns 2 (ctr1)		udp_send_skb	+10.5	
9	ns 2 (ctr1)	eth1	tx dev_queue_xmit	+11.8	
—— netns boundary ——					
10	ns 1 (init)	eth1	rx netif_receive_skb	+13.2	
11	ns 1 (init)		ip_output	+14.1	
12	ns 1 (init)	ens3	tx dev_queue_xmit	+15.0	

—— end of tracked events (12 hops, 15.0 μs total) ——

[p]rev packet [n]ext packet [f]ilter [s]ummary view [q]uit

MOCK-UP

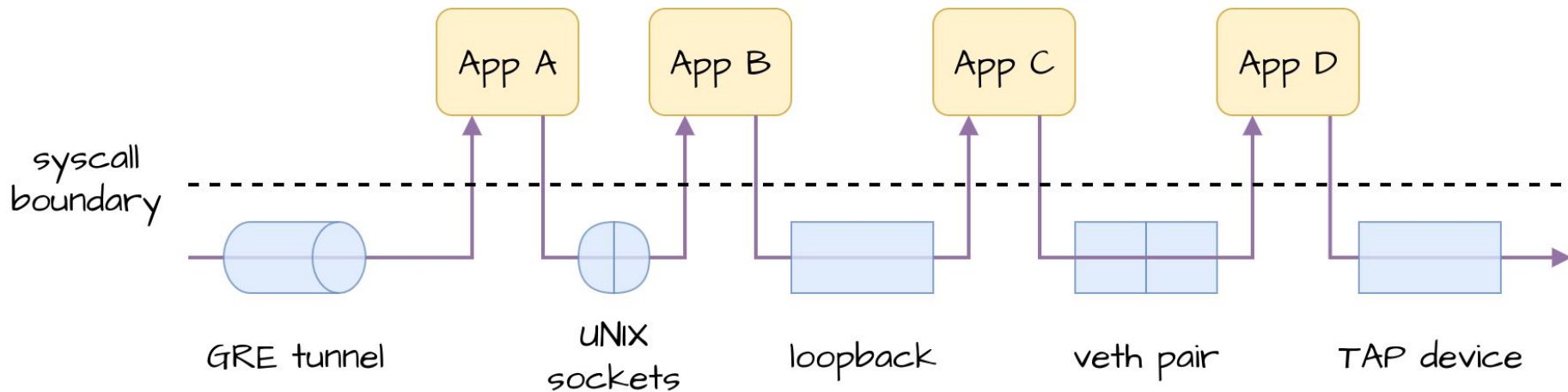
What do you use for tracing packets?





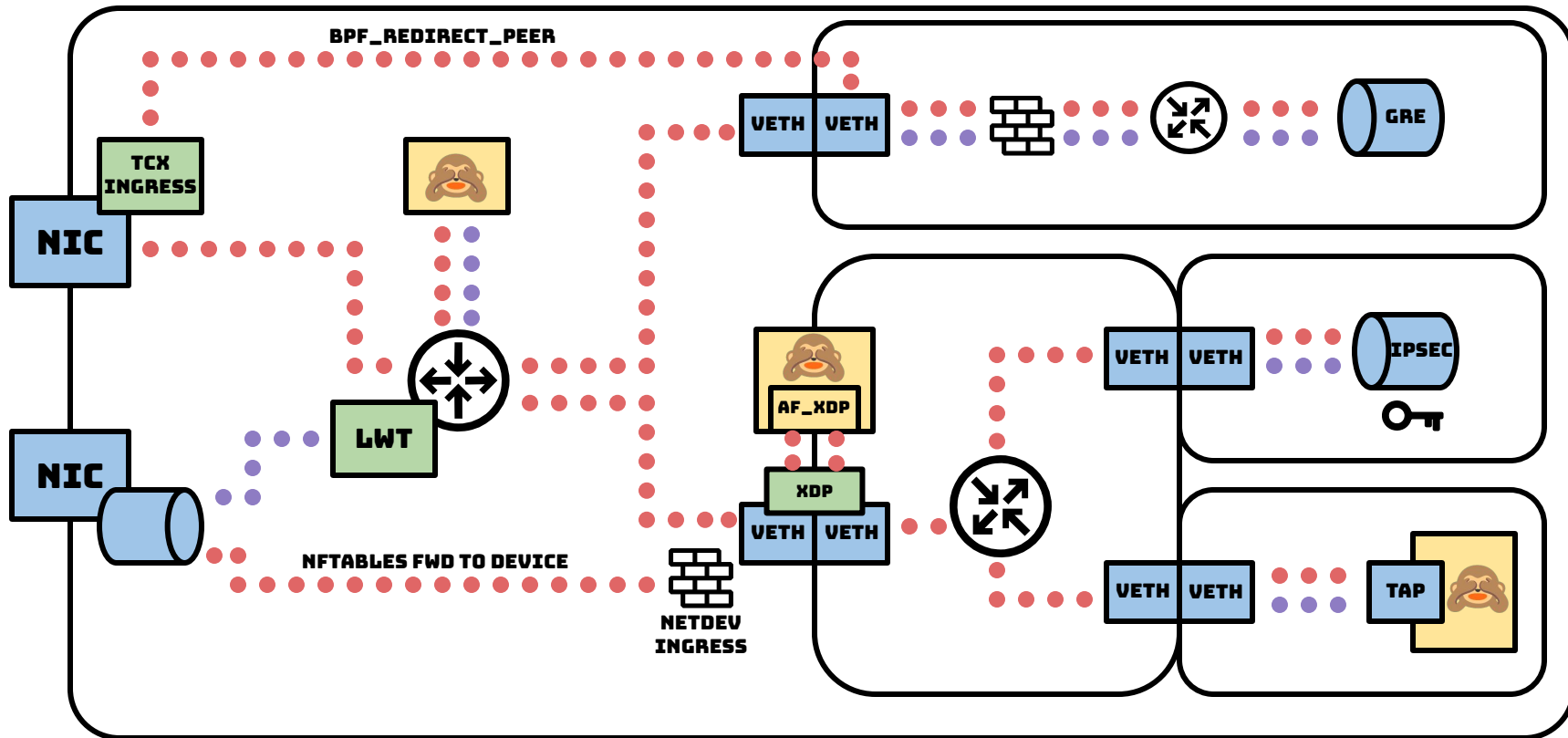
**If skb tracing is a solved problem,
what do you need BPF metadata for?**

Tracing L7 flows

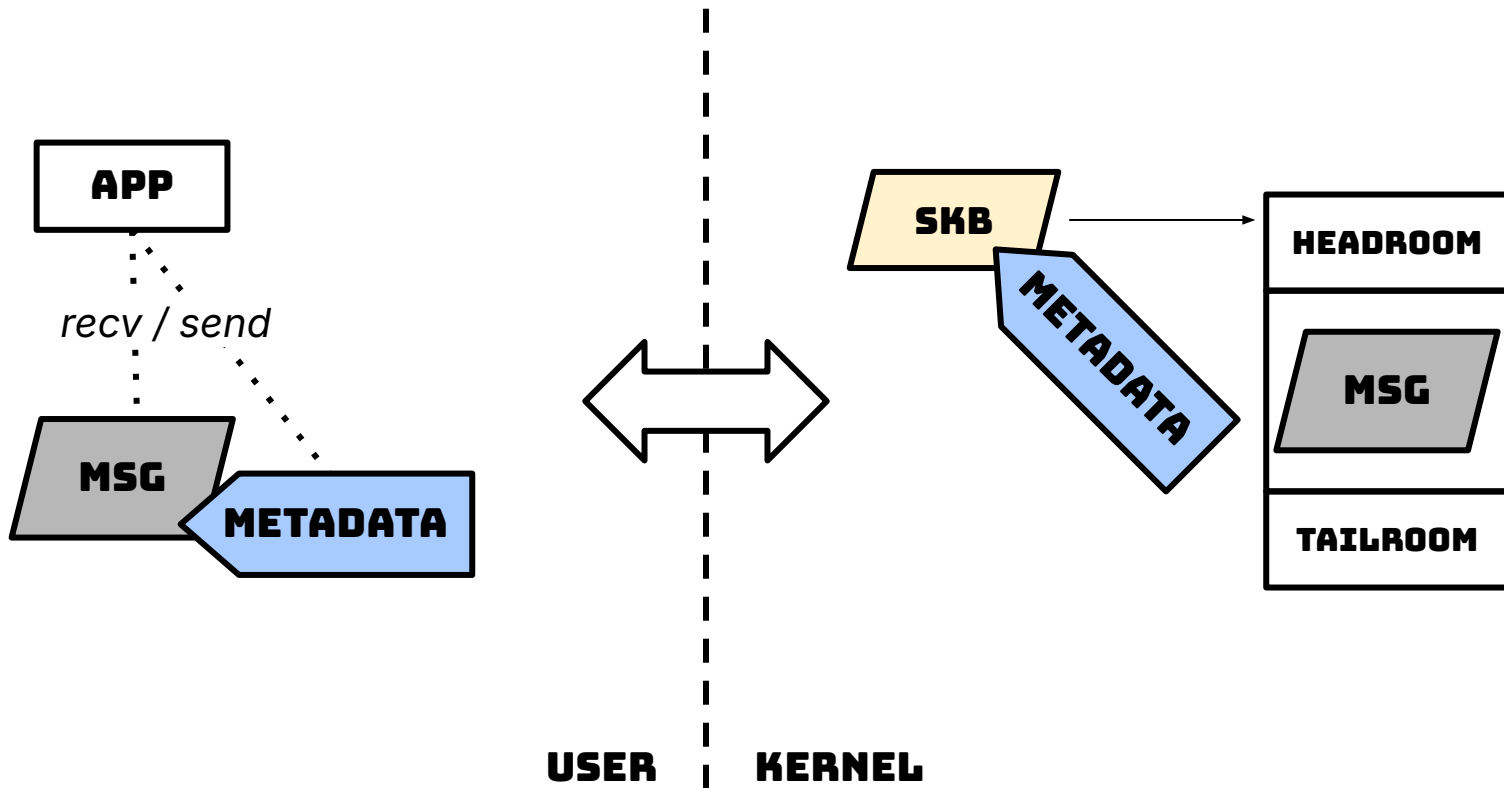




 We have blind spots today

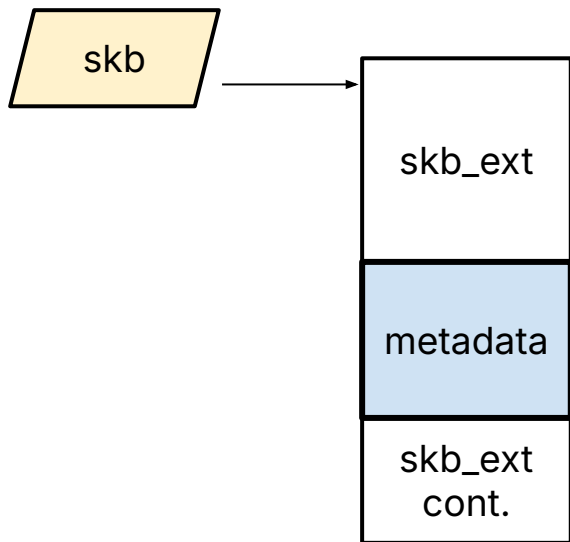


Idea – Apps recv/send metadata together with msgs

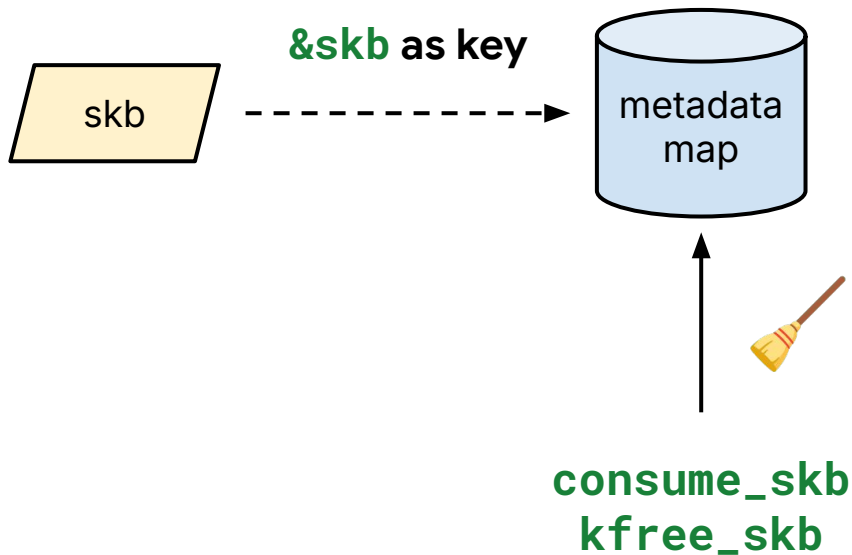


We had two options on the table...

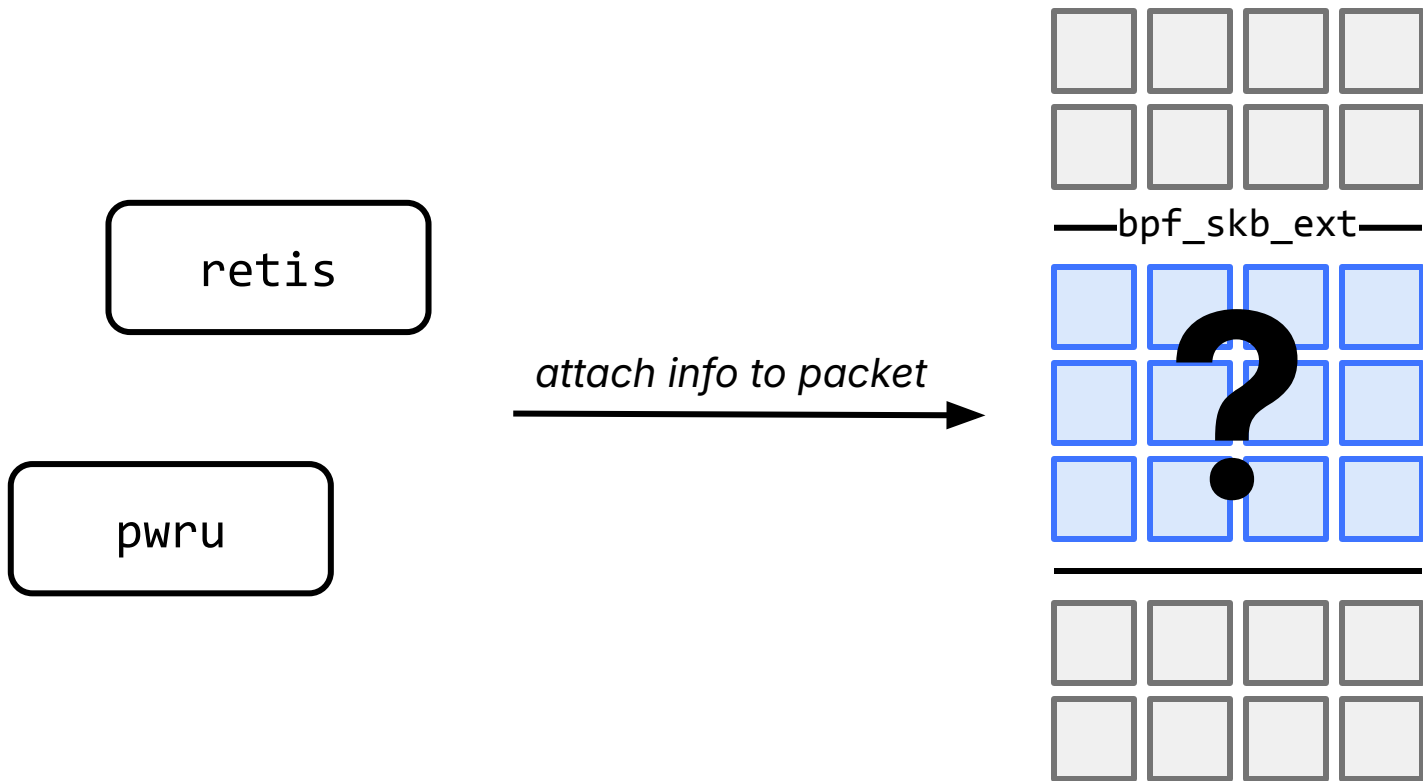
Option 1



Option 2



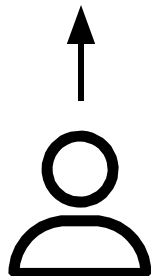
Option 1 – Challenges for tracing



Option 1 – Access to shared space is tricky

🤔 Make user specify
where to place metadata?

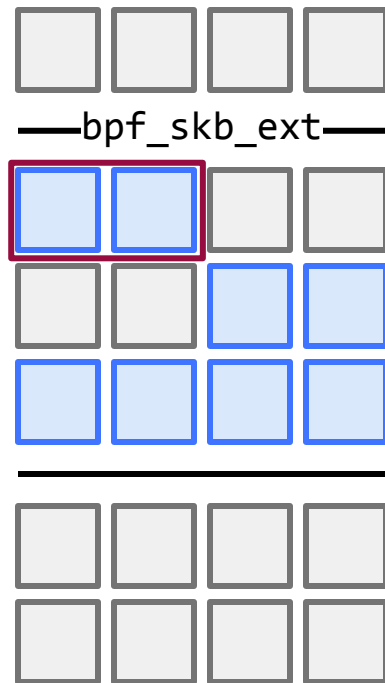
retis



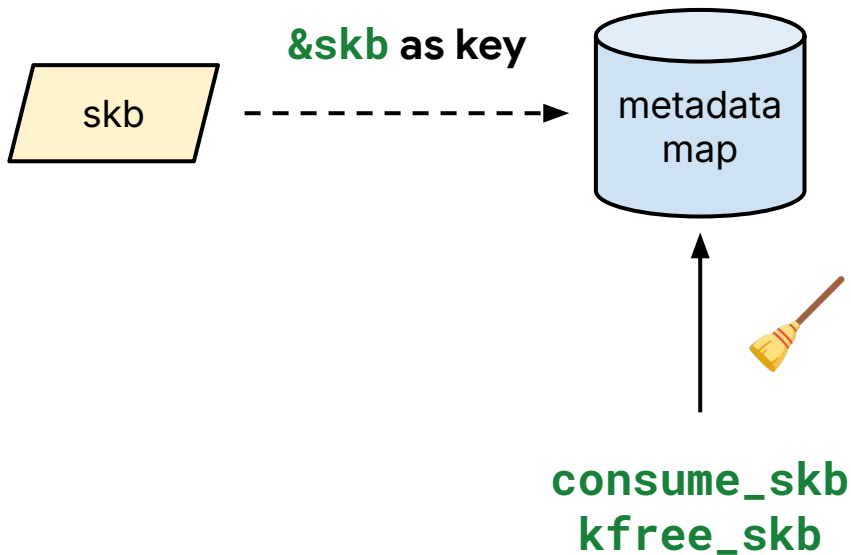
user

use
(**off=64, len=2**)
for metadata

attach info to packet
→
@ offset=64, length=2



Option 2 – has overhead issues



BPF tracing prog
for metadata cleanup
gets called for **every** skb

Option 2 – has overhead issues

Option 1

Tubular BPF execution time (CPU%) - test group 2 (923m16-18)



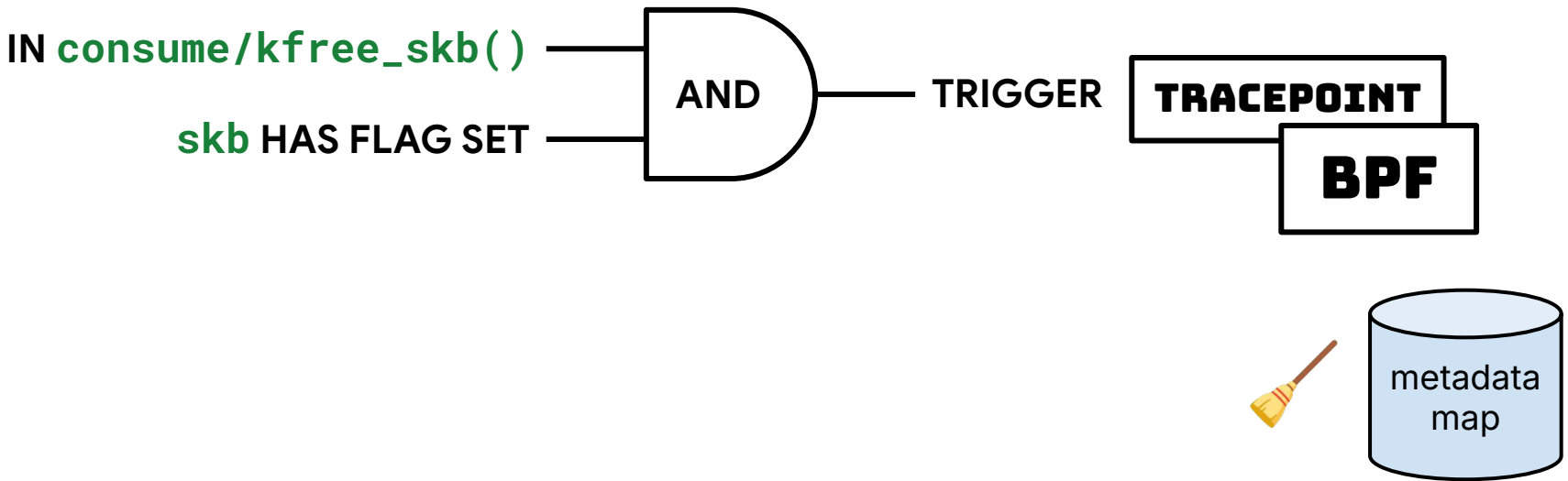
Option 2

Tubular BPF execution time (CPU%) - test group 1 (923m13-15)



metadata cleanup in [consume/kfree_skb](#)

Enter Option 3 – Gated skb tracepoints



Option 3 – Pay the cost but only for **skbs** you choose

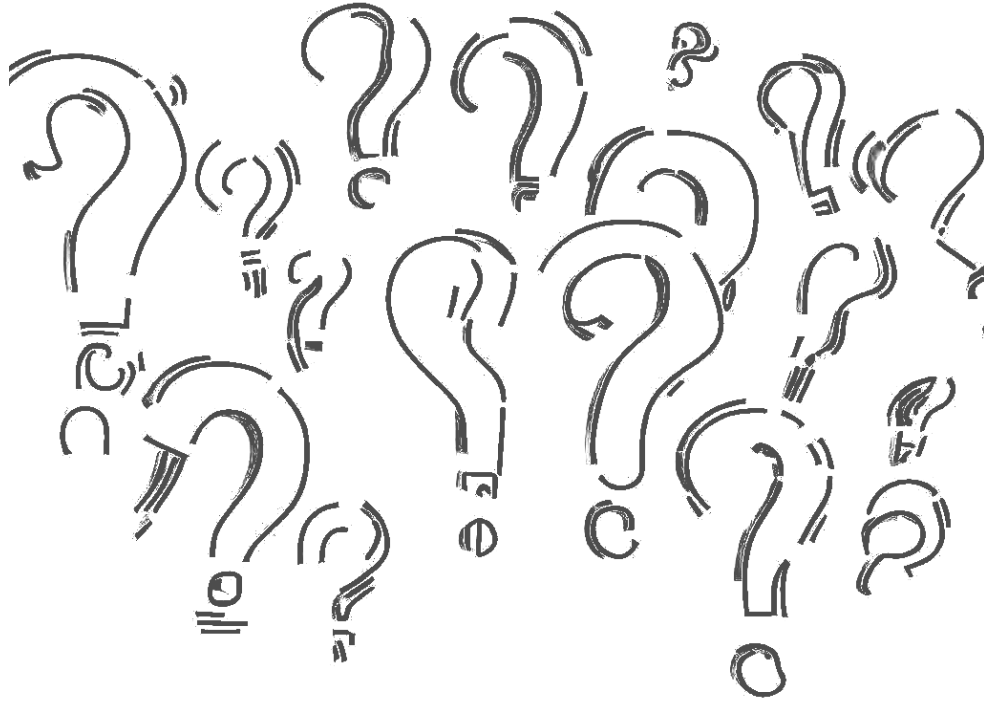


Runtime comparison for insert + lookup + delete sequence

variant	BPF ns/pkt
skb ext BPF dynptr (teardown not accounted)	264 ± 2
gated skb tracepoints (rhash, patched for min_size)	921 ± 35
gated skb tracepoints (rhash, vanilla)	1191 ± 44

Full breakdown at <https://github.com/jsitnicki/skb-metadata-bench/blob/main/REPORT.md#results>

Which solution one would you use?



Thank you

1. **[PATCH net-next v2 00/14] skb extension for BPF metadata (Sep 10)**
<https://msgid.link/20260910-bpf-meta-inside-skb-ext-v2-0-0b21e42180b0@cloudflare.com/>
2. **[PATCH net-next 00/15] Gated tracepoints for skb lifecycle (free+clone) (Sep 28)**
<https://msgid.link/20260928-bpf-meta-gated-tracepoints-v1-0-844dbf3e1edf@cloudflare.com/>

 jakub@cloudflare.com

Image Attribution

- [A bunch of question marks, Wikimedia Commons, CC BY-SA 4.0 DEED](#)
- [Hands holding a package, Vectorportal.com, CC BY 4.0](#)